KARNAVATI UNIVERSITY
NAAC GRADE A+

KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"C-LIKE LANGUAGE LEXICAL ANALYZER WITH ERROR REPORTING"

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Risheek Verma	23BTAI49
Rudrakumar Chopda	23BTAI50
Utpal Sakhiya	23BTAI51

Program: B.Tech CSE-AIML
Semester: VII

Under the Guidance of

Ms Neha Minocha
Assistant Professor

Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

1. Abstract 1

2. Introduction 1

2.1 Problem Statement and Objectives 1

3. Overview of Project 2

3.1 Architecture 2

3.2 Tokens Recognised 2

3.3 How the Scanner Works 3

3.4 Error Detection and Reporting 3

4. Technology Used 4

5. Flow Chart of the Project 5

6. Screen Shots of the Project 6

7. Results and Conclusion 7

References 7

1. Abstract

Lexical analysis is the first phase of a compiler: it reads the source program character by character and groups the characters into tokens such as keywords, identifiers, constants and operators. This project, **"C-Like Language Lexical Analyzer with Error Reporting"**, is a hand-written lexical analyzer for a C-like language, implemented in C++17.

The analyzer recognises nine categories of tokens, skips comments and builds a symbol table of identifiers. Its main contribution is error reporting: it detects eleven kinds of lexical errors and three kinds of warnings, reports each one with its line and column, the source line, a caret under the faulty text and a hint on how to fix it, and uses panic-mode recovery so that all errors in a program are reported in a single run. On the sample programs it produced 179 tokens with no errors for a valid program and found all 15 errors and 2 warnings in a faulty one; all 20 test cases passed.

2. Introduction

A compiler works in phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimisation and code generation [1]. The lexical analyzer (scanner) is the only phase that reads the raw source text. It converts the text into a stream of **tokens**, removes whitespace and comments, records line numbers for later error messages and enters identifiers into the symbol table. Each token has a **pattern**, usually a regular expression, and the actual text that matches it is called a **lexeme**.

A **lexical error** is a sequence of characters that matches no token pattern, for example an illegal character such as `@`, a string that is not closed, or a number such as `3.14.15`. Many simple scanners stop at the first such error or print a short message like "invalid token at line 6", so the programmer has to fix errors one at a time without being told what is wrong.

2.1 Problem Statement and Objectives

The aim of this project is to build a lexical analyzer that produces the correct tokens and symbol table for valid programs, and that also finds every lexical error in a single pass and explains each one clearly. The objectives were:

1. Implement a single-pass, hand-written scanner for a C-like language in C++17.

2. Recognise all common C token classes, including integer constants in three bases, floating-point constants, escape sequences and multi-character operators.
3. Build a symbol table recording each identifier's first position, number of uses and lines.
4. Detect at least ten classes of lexical errors and report each with position, caret and hint.
5. Recover from errors so that the whole program is checked in one run.

3. Overview of Project

3.1 Architecture

The program has three modules (Figure 1). `main.cpp` provides the menus and command-line mode and obtains the source code. The `Lexer` class in `lexer.cpp` scans the code once and returns a `LexResult` holding the tokens, the diagnostics (errors and warnings) and the symbol table. `report.cpp` formats this result for the screen or a text file. The lexer itself prints nothing, so it could be reused as the front end of a parser.

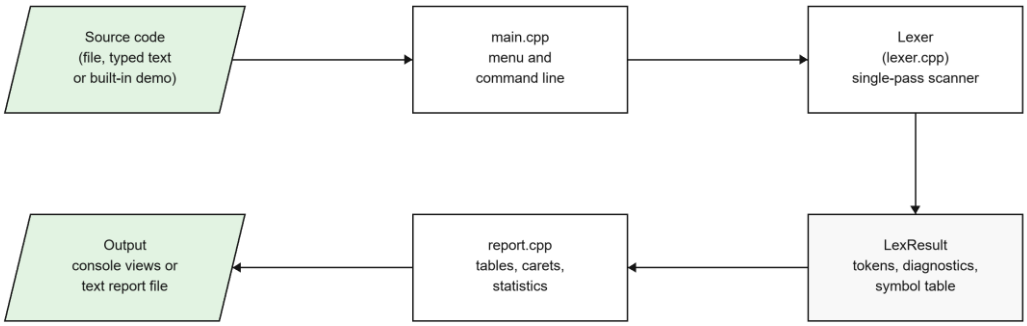


Figure 1: System architecture

3.2 Tokens Recognised

Table 1: Token categories

Category	Examples
KEYWORD (37 reserved words)	int, while, return, typedef, bool
IDENTIFIER	count, factorial, _tmp
INTEGER_CONST (decimal, octal, hex, U/L suffixes)	10, 0755, 0x1F, 0UL
FLOAT_CONST (fraction, exponent, F/L suffix)	3.0, 2.5e-1, 1.0f
CHAR_CONST and STRING_LITERAL (with escapes)	'A', '\n', "Total: %d\n"
OPERATOR (longest match)	+, <=, &&, <<=, ->
PUNCTUATOR	( ) { } [ ] ; , ...

Category	Examples
PREPROCESSOR (whole line)	#include <stdio.h>

3.3 How the Scanner Works

The scanner reads the input once from left to right, using `peek()` to look ahead and `advance()` to consume a character while updating the line and column. The first one or two characters of each lexeme decide which scanner function runs: a letter or underscore starts an identifier or keyword, a digit starts a number, a quote starts a string or character constant, and so on (see Figure 2). Operators use the **longest-match rule**, so `<<=` is one token rather than `<<` and `=`. Identifiers are entered into a hash-map-based symbol table, and a stack of open brackets is kept to detect unbalanced brackets.

Numbers are read in full before they are checked, so a mistake such as `2ndValue` is reported once for the whole word instead of being split into the number 2 and an identifier. Integer constants are also range-checked: anything above 18446744073709551615 is rejected.

3.4 Error Detection and Reporting

Table 2: Errors and warnings detected

Code	Problem	Example
E101	Invalid character	int price = 100$;
E102 / E103	Unterminated string / character constant	"Hello 'a
E104	Unterminated block comment	/* never closed
E105	Malformed number	2ndValue, 3.14.15, 0789, 0x, 1.5e
E106	Invalid escape sequence	"\q"
E107 / E109	Empty / multi-character constant	" 'abc'
E108	Identifier longer than 31 characters	thisIdentifierNameIsWayTooLong...
E110	Integer constant too large	99999999999999999999
E111	Unknown preprocessor directive	#defne
W201 / W202	Mismatched / unclosed bracket	if (a > b)) {
W203	/* inside a block comment	/* a /* b */

Every problem is reported in the same format, modelled on GCC and Clang:

```
error[E105] line 7, col 19: Too many decimal points in number '3.14.15'
      7 |     float ratio = 3.14.15;
        |                   ^~~~~~
    hint: A numeric literal can contain at most one decimal point.
```

After an error the scanner records the diagnostic, marks the lexeme as an INVALID token, skips the faulty text and carries on. This **panic-mode recovery** means the rest of the program is still scanned normally, so all errors are found in one run. Warnings do not count as failures: in command-line mode the program exits with code 1 only if there are errors.

4. Technology Used

- **C++17** with the standard library only: `std::string`, `std::vector`, `std::unordered_set` (keywords), `std::unordered_map` (symbol table), `std::stable_sort` (ordering diagnostics) and `strtoull` (range check).
- **g++** (tested with g++ 16.1 from MSYS2/MinGW-w64 on Windows 11), with `-std=c++17 -Wall -Wextra -O2`; it compiles with no warnings. The Windows build is statically linked, so `lexer.exe` runs on any PC without installing anything.
- **build.bat** (Windows) and a **Makefile** (Linux/macOS) for building; **Visual Studio Code** as the editor.
- A hand-written scanner was chosen instead of a generator such as Flex because it gives full control over error messages and recovery and shows clearly how the theory becomes code.

5. Flow Chart of the Project

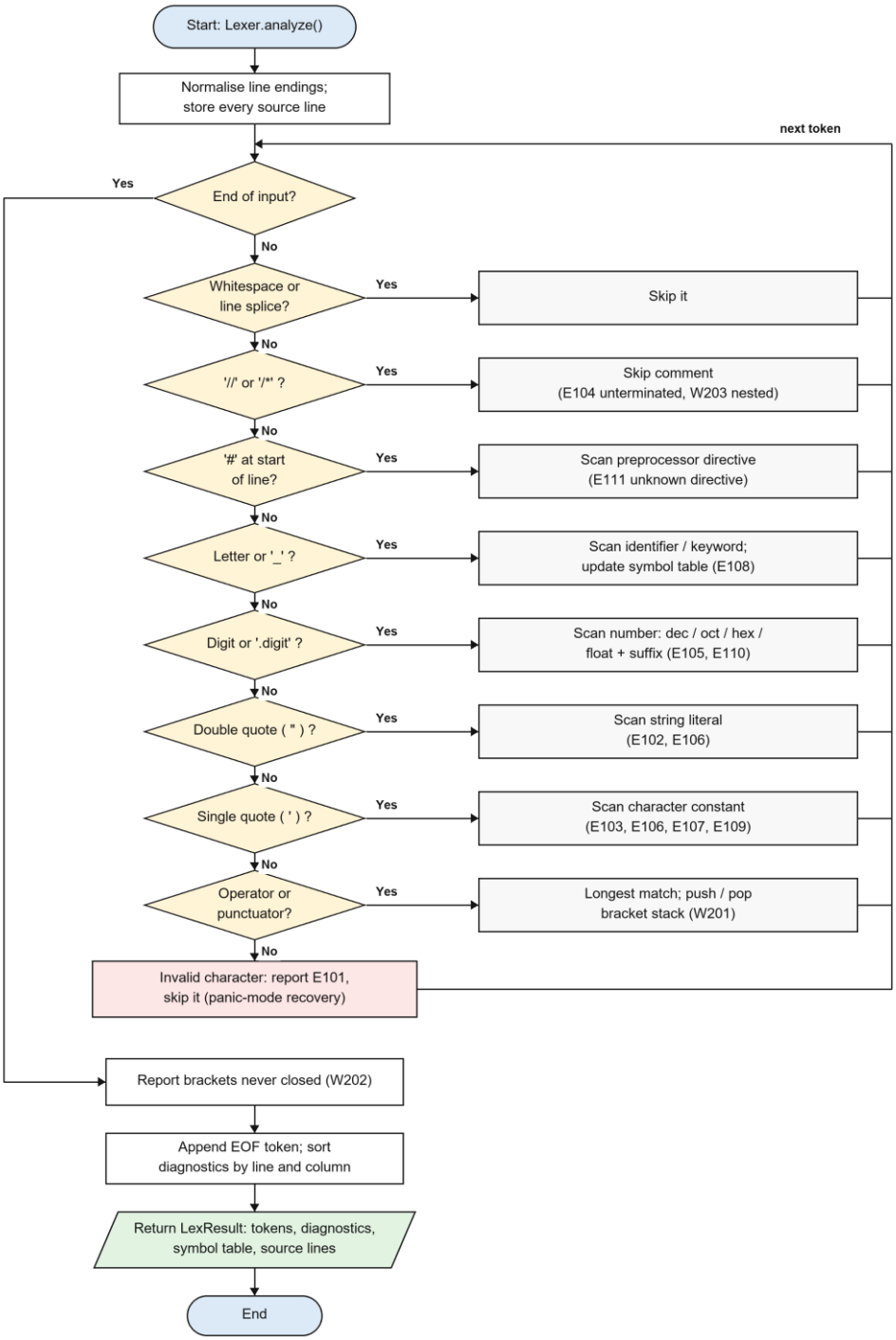


Figure 2: Flow chart of the lexical analysis loop

6. Screen Shots of the Project

Figure 3 shows code typed directly into the analyzer (menu option 2): both errors are found, each with its position, a caret and a hint. Figure 4 shows the symbol table produced for the valid demo program.

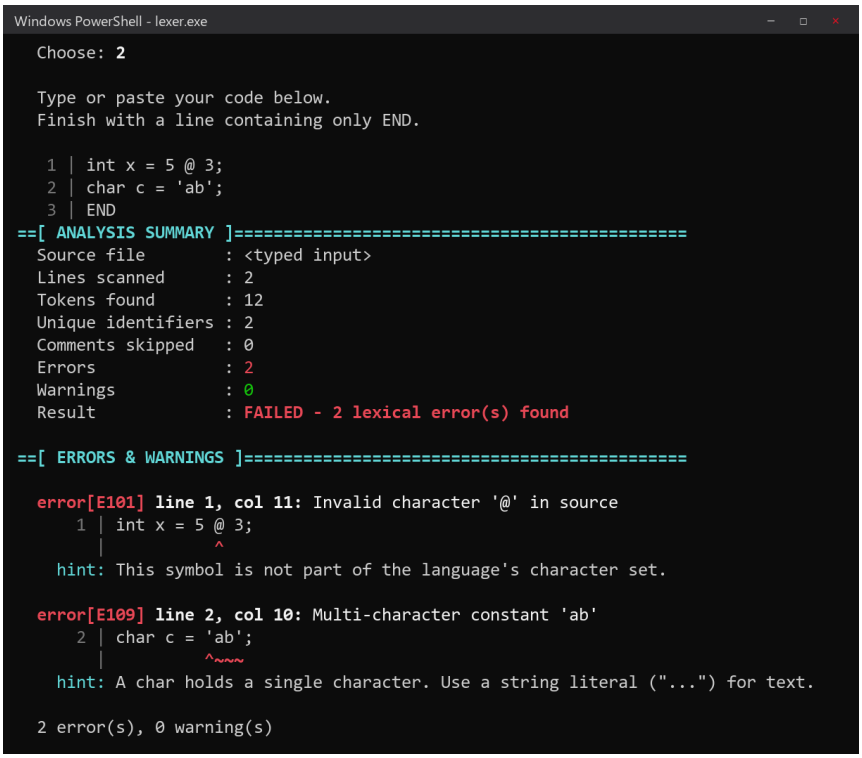


Figure 3: Errors reported for code typed by the user

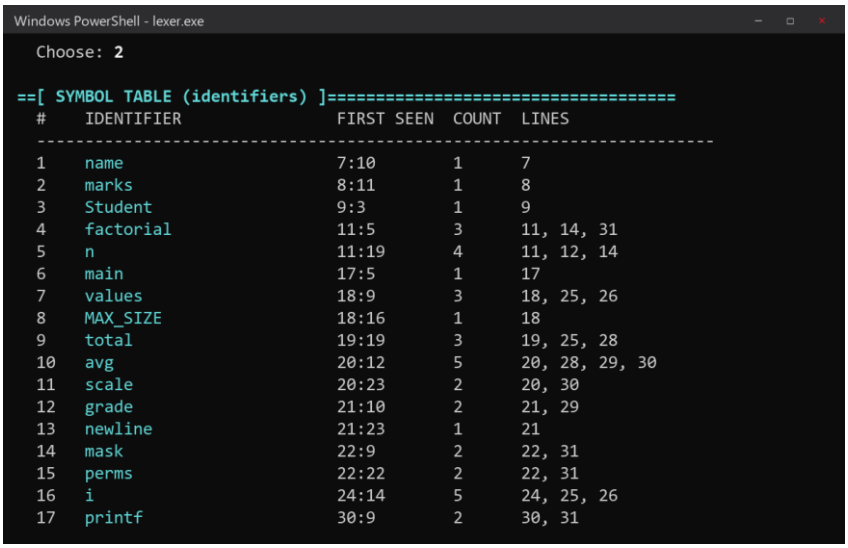


Figure 4: Symbol table for the valid demo program

7. Results and Conclusion

Table 3: Results on the sample programs

Measure	valid_program.c	program_with_errors.c
Lines / tokens	33 / 179	24 / 92
Unique identifiers / comments	17 / 2	15 / 10
Errors / warnings	0 / 0	15 / 2
Result	PASSED	FAILED (all problems found at the correct position)

In addition, 20 small test cases were run through the command-line mode, each targeting one feature or one error code (for example `x <<= 2;`, `int b = 09;`, `char c = 'a;` and an integer one larger than the maximum). All 20 gave the expected tokens, error codes and exit status, so every error and warning code was exercised at least once.

Limitations. The project covers the lexical phase only, so mistakes that need the grammar (such as a misspelt keyword used as a name) are not detected. Some rare C forms (wide-string prefixes, hexadecimal floats, trigraphs) are not supported, and macros are not expanded.

Conclusion. The project delivers a working lexical analyzer for a C-like language that tokenises programs correctly, builds a symbol table and, above all, reports lexical errors precisely and helpfully, finding all of them in one run. All objectives were met. Building it by hand put into practice the course concepts of tokens, patterns, regular definitions, finite automata, the longest-match rule, symbol tables and error recovery. A natural next step is to add a parser that uses this lexer as its front end.

References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson / Addison-Wesley, 2006.

[2] ISO/IEC 9899:2018, Information technology – Programming languages – C. ISO, 2018.